



**Mukesh Patel School of Technology Management &
Engineering (Mumbai Campus)**

IT DEPARTMENT

Project Synopsis

Title of Project:

**Adversarial Attack Detection for Large Language Models
(LLMs)**

Team Members:

Sumit Pandey | Dhruv Rathod | Rajpreet Khurana

Under the supervision of

Dr. Ruchi Sharma

Table of Contents

1. Title of the Project: Adversarial Attack Detection for Large Language Models(LLMs)	2
2. Team Members	2
3. Introduction	2
4. Literature Survey	4
5. Why is this particular topic chosen?	7

6. Objective and Scope of the project	8
7. Hardware and Software Requirements	9
7.1. Hardware (Minimum Requirements)	9
7.2. Software	9
7.2.1. Frontend	9
7.2.2. Backend	9s
7.2.3. Additional	9
8. Final Deliverables	10
9. References	10

1. Title of the Project: Adversarial Attack Detection for Large Language Models (LLMs)

2. Team Members

- 1) **Sumit Pandey (K044)**
- 2) **Dhruv Rathid (K055)**
- 3) **Rajpreet Khurana (K033)**

3. Introduction

The rapid integration of Large Language Models (LLMs) such as GPT-4, Claude, and LLaMA into customer support systems, code

assistants, healthcare triage tools, and enterprise chatbots has transformed how organisations interact with data and users. However, this widespread deployment has simultaneously exposed a new and rapidly growing attack surface. Unlike traditional software vulnerabilities that are exploited through code injection or buffer overflows, LLMs are vulnerable to adversarial manipulation of natural language itself — inputs that are crafted, often algorithmically, to bypass safety alignment and coerce a model into producing harmful, biased, or unauthorised outputs.

Researchers have demonstrated that carefully optimised adversarial suffixes appended to an otherwise harmless-looking prompt can reliably cause aligned, safety-trained models to comply with harmful requests, and that such suffixes transfer across different model families and vendors, including proprietary systems accessed only through an API. Beyond these gradient-based jailbreaks, a parallel and equally serious class of attack — prompt injection — allows an adversary to hijack the intended behaviour of an LLM-integrated application by embedding malicious instructions inside untrusted data that the model processes, such as a web page, an email, or a retrieved document. As LLMs are increasingly wired into agentic pipelines with tool access, file systems, and browsing capability, the consequences of an undetected adversarial input escalate from an embarrassing generation to a genuine security incident involving data exfiltration, unauthorised transactions, or system compromise.

Despite the seriousness of this threat, most production deployments still rely on ad-hoc, static keyword filters or the base alignment of the underlying model, both of which have been repeatedly shown to be brittle and easily evaded through paraphrasing, encoding tricks, role-play framing, and optimisation-based search. Recognising this gap, we propose the development of an Adversarial Attack Detection tool for LLMs — a model-agnostic, pluggable security layer capable of identifying jailbreak attempts, prompt injection payloads, and other adversarially perturbed inputs before they reach a production LLM, and of flagging anomalous or unsafe outputs before they reach an end

user. By combining statistical signal analysis (such as perplexity and token-distribution anomalies), embedding-based semantic classifiers, and lightweight guard models, our solution aims to give developers and organisations a practical, low-latency way to harden their LLM-integrated applications against an evolving and increasingly automated class of attacks, without needing to retrain or fine-tune the underlying foundation model itself.

4. Literature Survey

The vulnerability of aligned language models to adversarial manipulation was formalised most influentially by Zou et al., who introduced the Greedy Coordinate Gradient (GCG) attack. GCG automatically searches for a short adversarial suffix that, when appended to a harmful query, maximises the probability that the model responds with an affirmative completion rather than a refusal. The attack combines greedy and gradient-based token search over a white-box surrogate model, and the resulting suffixes were shown to transfer with high success rates not only across open-source models such as Vicuna and LLaMA-2-Chat, but also to public interfaces of closed, proprietary systems including ChatGPT, Bard, and Claude, despite none of these systems being directly optimised against during the attack. This transferability result was significant because it demonstrated that an attacker with no access to a target model's weights could still mount an effective, automated jailbreak, fundamentally challenging the assumption that black-box deployment alone provides meaningful protection.

Follow-up work has continued to refine both the offensive and interpretability aspects of these attacks. AutoDAN, for instance, generates adversarial prompts that remain fluent and human-readable, allowing them to bypass simple perplexity-based filters that flag the gibberish-like suffixes typical of GCG. More recent work exploring Chain-of-Thought-guided gradient search (CoT-GCG) has shown that reasoning-augmented optimisation can further improve attack transferability while operating under constrained computational

budgets, and that safety classifiers such as Llama Guard reveal systematic weaknesses in current alignment pipelines for specific categories of harmful content. Collectively, this body of work establishes that jailbreak generation is no longer a manual, ad-hoc process reliant on human ingenuity, but an increasingly automated optimisation problem — raising the urgency for equally automated, generalisable detection mechanisms.

A second major thread of research addresses prompt injection, a distinct but related threat first characterised by Perez and Ribeiro, who showed that LLMs can be misled by simple, crafted inputs embedded within otherwise legitimate data, resulting in goal hijacking and system prompt leakage. Because LLM-integrated applications frequently concatenate trusted developer instructions with untrusted third-party content (web pages, documents, emails, tool outputs), an attacker can smuggle instructions directly into the data channel, a problem that has no direct analogue in traditional software security and is difficult to fully resolve through input sanitisation alone.

On the defensive side, the literature broadly separates mitigation strategies into prevention-based and detection-based approaches. Prevention-based defences attempt to pre-process instructions and data — for example through the delimiter-based separation of prompts and data explored by Yi et al., or fine-tuning the base model to be more resistant to injected instructions — but such techniques have shown only limited effectiveness against novel or adaptive attacks and often degrade general model utility. Detection-based defences, which are the primary focus of this project, instead treat the incoming prompt (or the model's response) as a signal to be classified as benign or adversarial. The earliest and most widely studied of these is perplexity-based filtering, proposed by Alon and Kamfonas and extended by Jain et al. through a windowed perplexity approach that computes perplexity over sliding segments of the prompt rather than the sequence as a whole, since optimisation-based suffixes such as those produced by GCG tend to be locally low-probability under a

reference language model even when the surrounding prompt is fluent.

While effective against gradient-based, gibberish-like suffixes, perplexity filtering alone generalises poorly: benchmarking work evaluating guardrails such as Azure Prompt Shield and Meta's Prompt Guard across multiple jailbreak styles has shown that score-based detectors are limited by a single global threshold and struggle to balance true-positive detection against false positives on legitimate, unusual-but-benign prompts. This has motivated a shift towards learned classifiers: embedding-based approaches encode a prompt using a pretrained language model and train a lightweight classifier (e.g., logistic regression, random forest, or a small transformer head) on top of the resulting representation, which has been shown to detect semantically disguised injection attempts that perplexity-based methods miss entirely. Purpose built guard models such as Meta's Prompt Guard, ProtectAI's DeBERTa-based classifiers, and InjecGuard extend this idea further, fine-tuning encoder models specifically on curated corpora of benign and adversarial prompts, with InjecGuard in particular reporting accuracy competitive with general-purpose LLM judges such as GPT-4o while remaining lightweight enough for low-latency deployment. A related line of work, GradSafe, analyses the gradients induced by a prompt paired with a compliant response to identify safety-critical patterns without requiring any additional fine-tuning of the target model.

Nevertheless, recent adversarial evaluations caution that no single defence is currently robust in isolation. An empirical study testing six prominent guardrail systems, including commercial offerings, found that character-level obfuscation and algorithmic adversarial machine learning evasion techniques could bypass detection with success rates approaching 100% in some configurations, underscoring that guardrails themselves constitute an attack surface subject to the same optimisation pressures as the underlying LLMs they protect. This finding directly motivates our project's approach of layering multiple complementary detection signals — statistical, embedding-based, and

LLM-judge-based — rather than depending on any single detector, and of continuously benchmarking the tool against an evolving adversarial test suite rather than treating detection as a solved, static classification problem.

5. Why is this particular topic chosen?

The past two years have seen an unprecedented rate of LLM adoption across customer-facing products, internal enterprise tools, and increasingly autonomous agentic systems that can browse the web, execute code, and take actions on a user's behalf. This shift dramatically raises the stakes of an undetected adversarial prompt: a jailbreak that once merely produced an inappropriate paragraph of text can now, in an agentic context, translate into an unauthorised file deletion, a leaked API key, or a fraudulent transaction. Security teams and AI safety researchers alike have identified adversarial robustness as one of the most urgent open problems in deploying LLMs responsibly, yet, unlike traditional cybersecurity domains such as network intrusion detection or malware analysis, tooling for LLM-specific threat detection remains immature, fragmented across research prototypes, and largely inaccessible to small teams without dedicated AI security expertise.

Existing commercial guardrail offerings are typically closed-source, tied to a single cloud vendor, or narrowly scoped to a single attack category such as toxic content moderation rather than the broader spectrum of jailbreaks, prompt injections, and adversarially optimised inputs. This creates a clear opportunity for a detection tool that is model-agnostic, transparent in its detection logic, and specifically designed to combine multiple complementary techniques so that the weaknesses of one detector (for instance, perplexity filtering's blindness to fluent, semantically disguised attacks) are compensated for by another (such as an embedding-based classifier or an LLM-based judge). Building this tool also allows us to engage directly with an actively evolving research area, working with real adversarial benchmarks and attack reproductions rather than static, historical

data, and to produce a system whose core detection techniques generalise well beyond any single LLM vendor or version.

6. Objective and Scope of the project

As organisations of every size race to embed LLMs into their products, the gap between the pace of adoption and the maturity of corresponding security tooling continues to widen. Startups and small engineering teams, in particular, often lack the dedicated AI red-teaming or security resources needed to continuously test their LLM-integrated applications against novel jailbreak and prompt injection techniques, leaving them exposed to reputational, financial, and data-privacy risks. Larger enterprises deploying LLMs in regulated or safety-critical contexts face an additional burden of demonstrating due diligence around model robustness as part of emerging AI governance requirements.

Our project's primary objective is to design and build a lightweight, pluggable detection layer that can be inserted into any LLM application pipeline — regardless of the underlying model provider — to screen incoming prompts and, optionally, outgoing completions for signs of adversarial manipulation. The tool combines three complementary detection strategies: (i) statistical analysis using windowed perplexity and token-distribution anomaly scoring to catch gradient-optimised, low-fluency suffixes such as those produced by GCG-style attacks; (ii) an embedding-based semantic classifier, fine-tuned on curated benign and adversarial prompt corpora, to catch fluent, semantically disguised jailbreaks and injection attempts that evade perplexity filtering; and (iii) an optional LLM-as-a-judge verification stage for high-risk or ambiguous inputs, providing an interpretable, natural-language rationale for flagged content. Detected threats are logged and surfaced through a dashboard so that developers can audit attack patterns over time and continuously refine detection thresholds.

The scope of the project targets three primary attack categories: (a) optimisation-based jailbreaks, including gradient-based adversarial suffixes and their more fluent variants; (b) prompt injection attacks, both direct (embedded in the user's own prompt) and indirect (embedded in third-party data such as documents, web pages, or tool outputs consumed by the model); and (c) role-play and persona-based social-engineering jailbreaks that attempt to bypass alignment through fictional framing. The tool is designed to be evaluated against open adversarial benchmarks such as AdvBench and publicly available jailbreak prompt collections, and to remain model-agnostic by operating purely on text inputs and outputs rather than requiring white-box access to model weights, making it deployable alongside both open-source models and closed, API-only commercial LLMs.

7. Hardware and Software Requirements

7.1. Hardware (Minimum Requirements)

- Quad-core Intel Core i5 or AMD Ryzen 5 (or equivalent)
- 16 GB RAM (recommended for local embedding/classifier inference)
- NVIDIA GPU with at least 6 GB VRAM (optional, for local model fine-tuning and faster inference)
- 100 GB free disk space
- Debian-based OS (e.g. Ubuntu) or Windows with WSL2
- Stable Internet Connection, 10 Mbps (for LLM API access)

7.2. Software

7.2.1. Frontend

- HTML5, CSS3
- React.js for the framework, with Tailwind CSS and Material UI for the dashboard
- Recharts / D3.js for attack analytics visualisation

7.2.2. Backend

- Python (FastAPI) for the detection engine and model-serving layer
- Node.js and Express for the application/API gateway layer
- PostgreSQL / MongoDB for logging detected prompts, scores, and audit trails
- Hugging Face Transformers and PyTorch for embedding models and classifier fine-tuning
- Redis for low-latency caching of detection scores
- Docker and Docker Compose for containerised deployment

7.2.3. Additional

- Visual Studio Code / PyCharm for IDE
- Git and GitHub for version control
- AWS / GCP for cloud deployment and GPU inference
- Weights & Biases (or MLflow) for experiment tracking during classifier training

8. Final Deliverables

- A curated benchmark dataset of benign, jailbreak, and prompt-injection prompts assembled from open adversarial corpora and reproduced attack generations (e.g., GCG-style suffixes), used for training and evaluating the detection models.
- A detection engine that scores incoming prompts (and optionally model outputs) using a combination of perplexity-based, embedding-based, and LLM-judge-based signals, exposed as a lightweight, model-agnostic API.
- A web-based dashboard allowing developers to monitor flagged prompts, inspect detection rationale, review false-positive/false-negative trends, and configure detection sensitivity.

- A server-side application acting as a middleware/proxy layer between an LLM-integrated application and the underlying LLM provider, routing traffic through the detection engine.
- A detailed evaluation report benchmarking the tool's detection accuracy, false-positive rate, and latency overhead against known jailbreak and prompt-injection attack suites, along with an analysis of its robustness to evasion attempts.

9. References

- [1] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and Transferable Adversarial Attacks on Aligned Language Models," arXiv preprint arXiv:2307.15043, 2023.
- [2] E. Perez and I. Ribeiro, "Ignore Previous Prompt: Attack Techniques For Language Models," arXiv preprint arXiv:2211.09527, 2022.
- [3] N. Jain et al., "Baseline Defenses for Adversarial Attacks Against Aligned Language Models," arXiv preprint arXiv:2309.00614, 2023.
- [4] Y. Liu et al., "Formalizing and Benchmarking Prompt Injection Attacks and Defenses," arXiv preprint arXiv:2310.12815, 2023.
- [5] H. Li et al., "InjecGuard: Benchmarking and Mitigating Over-defense in Prompt Injection Guardrail Models," arXiv preprint arXiv:2410.22770, 2024.
- [6] W. Hackett, L. Birch, S. Trawicki, N. Suri, and P. Garraghan, "Bypassing LLM Guardrails: An Empirical Analysis of Evasion Attacks against Prompt Injection and Jailbreak Detection Systems," arXiv preprint arXiv:2504.11168, 2025.

- [7] R. Panchendrarajan et al., "Adversarial Prompt Evaluation: Systematic Benchmarking of Guardrails Against Prompt Input Attacks on LLMs," arXiv preprint arXiv:2502.15427, 2025.

Mentor Signature

2025-2026

Dr. Ruchi Sharma